Senior Data Platform Engineer | Shanghai, China | Open to Global Remote | jromeguan@gmail.com |
https://linkedin.com/in/jrome-guan-47888241a | https://github.com/jlty258/portfolio

Guan Chen

Professional Summary

Platform engineer with 13+ years of experience building distributed data infrastructure across analytical computing, streaming systems, and privacy computing. Career progression from scientific computing through enterprise analytical platforms, large-scale streaming infrastructure, and privacy-preserving execution systems.

Designed and owned platform architectures serving PB-scale analytical workloads, 10B+ daily telemetry events across 1M+ connected vehicles, and distributed privacy computing execution pipelines. Focus on reusable abstractions, execution planning, storage layers, and platform standardization that enable product teams to move faster.

The through-line across all work: build platforms that abstract complexity behind stable interfaces, not one-off pipelines that solve a single use case.

Core Expertise

- Distributed Systems and Platform Engineering
- Streaming Infrastructure (Kafka, Flink)
- Analytical Computing (Hive, Spark, Elasticsearch)
- Privacy Computing and Execution Planning
- Storage Abstraction and Metadata
- Data Governance and Platform Standardization

Professional Experience

Senior Data Platform Engineer

Shanghai Puxin Future Internet Research Institute | Dec 2024 – Present

ChainWeaver / MIRA Privacy Computing Platform

Architected the execution layer for a privacy computing platform that transforms declarative queries into distributed execution plans across heterogeneous compute and storage backends.

Execution Planning

Designed the PQL execution planning pipeline that converts platform queries into optimized distributed DAGs. Defined planner interfaces that separate query semantics from execution strategy, allowing the platform to evolve execution engines without changing upstream query contracts.

Query Planner

Built the query planner responsible for logical plan generation, operator rewriting, and cost-aware execution graph construction. The planner abstracts privacy-preserving computation patterns so application teams submit declarative queries rather than orchestrating distributed jobs manually.

Distributed DAG

Defined the distributed DAG model that represents cross-party computation workflows. Established DAG semantics for dependency resolution, stage boundaries, and parallel execution across execution engines.

Data Service

Designed a reusable Data Service layer that provides unified data access across storage backends. Abstracted read/write paths behind stable interfaces so execution engines consume data without binding to specific storage implementations.

Storage Abstraction

Established storage abstraction interfaces that decouple execution logic from physical storage. Enabled the platform to support multiple storage backends through a consistent contract rather than per-backend integration code.

Benchmark Platform

Built a benchmark platform to measure execution planning and runtime performance across query patterns and hardware configurations. Provided measurable evidence for optimization decisions rather than ad-hoc performance tuning.

Platform Standardization

Defined platform interfaces and engineering standards across planner, scheduler, execution engine, and data service components. Reduced integration cost for new execution backends and improved cross-team delivery consistency.

Architecture

```
PQL → Planner → Execution DAG → Scheduler → Execution Engine → Data Service → Storage
```

Engineering Decisions

Chose a dedicated query planner over template-based job configuration because privacy computing queries vary in structure. A planner adapts execution graphs to query semantics; templates require manual updates for each new pattern.

Introduced storage abstraction between execution engines and physical storage because different deployments use different backends. The trade-off is additional interface indirection, mitigated by keeping the Data Service contract thin.

Built a benchmark platform rather than relying on ad-hoc profiling because execution planning optimization involves multiple dimensions — planner logic, DAG structure, engine selection, storage access — that cannot be tuned correctly without measurement.

Impact

Application teams submit declarative PQL queries instead of orchestrating distributed jobs manually. Data Service supports 10+ heterogeneous data source types, billion-scale table reads, and S3 object storage access. New execution backends integrate through documented platform interfaces. Optimization decisions are driven by benchmark evidence rather than assumptions.

Technologies: Java, Apache Flink, Docker, Kubernetes, Linux

Senior Data Platform Engineer / Technical Architect

Shanghai Jiayu Intelligent Robotics | Jun 2020 – Sep 2024

Connected Vehicle Platform

Owned the streaming data platform supporting connected vehicle telematics at national scale. Designed end-to-end architecture from vehicle ingestion through real-time processing to business-facing APIs.

Scale

- 1M+ connected vehicles
- 10B+ telemetry events per day
- 1000+ vCore streaming cluster

Streaming Platform

Architected the streaming platform that ingests, processes, and serves vehicle telemetry data. Established platform boundaries between ingestion, stream processing, storage, and serving layers so each component could scale independently.

Kafka Ingestion Layer

Designed Kafka-based ingestion to decouple vehicle T-Box producers from downstream consumers. Partitioning and topic design accommodated burst traffic from millions of devices while maintaining ordering guarantees where required.

Apache Flink Processing

Built Flink-based stream processing pipelines for real-time aggregation, enrichment, and routing of telemetry events. Separated computation from storage so processing logic could evolve without rewriting ingestion infrastructure.

Real-time Warehouse

Established a real-time warehouse layer on Apache Doris to serve low-latency analytical queries over streaming-derived datasets. Chose Doris for its columnar storage and query performance on high-volume time-series workloads.

Metadata and Data Governance

Defined metadata models and data governance standards across the streaming platform. Standardized schema registration, lineage tracking, and data quality rules so downstream teams could discover and trust telemetry datasets.

Dinky Platform

Integrated and extended the Dinky platform for Flink job management, monitoring, and operational workflows. Reduced operational overhead for streaming job deployment across the 1000+ vCore cluster.

Architecture

```
Vehicle T-Box → Kafka → Apache Flink → Apache Doris → Business APIs
```

Engineering Decisions

Selected Kafka as the mandatory ingestion boundary because at 1M+ device scale, direct coupling between T-Box producers and processing systems creates cascading failures during traffic bursts. The trade-off is Kafka cluster operational overhead, justified at 10B+ events/day.

Chose Apache Flink for stream processing because stateful computation with exactly-once semantics is required for aggregation workloads at this scale. Operational complexity on the 1000+ vCore cluster is mitigated through Dinky platform integration for job lifecycle management.

Selected Apache Doris as the real-time warehouse layer rather than serving directly from Flink state because business APIs require ad-hoc analytical queries over historical and real-time data. Columnar storage provides query performance for time-series patterns. The trade-off is additional data movement latency from Flink to Doris, acceptable for business query patterns tolerating seconds-level freshness.

Established metadata and data governance as platform primitives rather than per-team conventions because at national device scale, schema discovery and data quality cannot rely on tribal knowledge.

Impact

Scaled Apache Flink platform from 10,000 to 1,000,000+ connected vehicles with stable production output. Platform ingests and processes 10B+ telemetry events/day without ingestion-producer coupling failures during peak traffic. Downstream teams discover and trust telemetry datasets through standardized metadata catalog. Flink job operations scale across 1000+ vCore cluster through integrated operational tooling.

Technologies: Kafka, Apache Flink, Apache Doris, metadata systems, data governance

---

Senior Data Platform Engineer

Ping An Securities | May 2017 – Apr 2020

Enterprise Analytical Computing / Metric Platform

Designed and owned the enterprise metric platform that transformed PB-scale Hadoop analytical workloads into searchable, dynamically schema-aware metrics served through Elasticsearch.

Scale

- 1000+ vCore analytical cluster
- PB-scale Hadoop storage
- 2000 Hive physical columns per table
- 100000+ Elasticsearch searchable fields

Metric Platform

Architected the enterprise metric platform that unified analytical computation with interactive search. Replaced ad-hoc Hive queries with a standardized metric definition and serving layer.

Dynamic Schema

Designed a dynamic schema expansion mechanism that mapped Hive analytical outputs to Elasticsearch index structures without manual schema migration. Solved the problem of 2000+ physical columns exceeding Elasticsearch field limits through a Map-based schema translation layer.

Hive Optimization

Optimized Hive query plans and storage layouts for PB-scale analytical workloads. Reduced query latency and resource consumption on the 1000+ vCore cluster through partition pruning, column pruning, and execution tuning.

Hadoop Optimization

Improved cluster utilization and job scheduling efficiency across the PB-scale Hadoop deployment. Established operational standards for resource allocation and workload isolation.

Elasticsearch Serving Layer

Built the Elasticsearch serving layer that exposed 100000+ searchable metric fields to business applications. Decoupled analytical computation in Hive from interactive search in Elasticsearch.

Architecture

```
Hive → Map → Dynamic Schema → Elasticsearch
```

Engineering Decisions

Separated analytical computation (Hive) from search serving (Elasticsearch) because batch computation and interactive search have incompatible latency and scaling requirements. Each layer optimizes independently. The trade-off is data freshness limited by batch schedule, acceptable for enterprise metric use cases.

Designed Map-based dynamic schema expansion rather than direct column-to-field mapping because 2000+ Hive physical columns must become 100000+ searchable Elasticsearch fields — direct mapping exceeds Elasticsearch per-index field limits. The Map translation layer provides indirection that scales metric count without manual per-metric index restructuring.

Invested in Hive and Hadoop optimization rather than migrating compute engines because existing PB-scale infrastructure investment and team expertise made optimization the lower-risk path to performance improvement.

Impact

Reduced 1000+ vCore batch computation time from 2.5 hours to 1.5 hours through Hive query optimization and Hadoop workload tuning. Business applications query 100,000+ searchable metric fields through Elasticsearch without direct Hive access or SQL expertise. New metrics onboard through the Map layer without manual schema migration across Hive and Elasticsearch.

Technologies: Hive, Hadoop, Spark, Flink, Elasticsearch, HDFS

Data Platform Engineer

Shanghai Jiayin Data Technology

Restaurant Intelligence / Commercial Wi-Fi Analytics

Built data processing pipelines for restaurant intelligence and commercial Wi-Fi analytics. Applied Storm for real-time event processing and Hive/Hadoop for batch analytical workloads in an early-stage data platform environment.

This role introduced streaming data processing patterns that later scaled to national vehicle telematics infrastructure. The domain — translating physical world signals (Wi-Fi presence, restaurant traffic) into analytical datasets — established the product-facing data platform mindset that defines subsequent career work.

Technologies: Storm, Hive, Hadoop

Research Engineer

Shanghai Ship and Shipping Research Institute

Hydrodynamic Simulation / Scientific Computing

Applied Hadoop and Hive to large-scale hydrodynamic simulation workloads in naval architecture and ocean engineering research. Translated scientific computing requirements — massive parallel numerical simulation — into distributed data processing infrastructure.

This work established the career foundation in distributed systems: partitioning large computational problems across cluster nodes, managing data locality, and optimizing batch processing throughput. The transition from scientific computing to enterprise data infrastructure was a natural progression from simulation-scale parallelism to analytical-scale platform engineering.

Also completed Master degree in Naval Architecture and Ocean Engineering at this institute.

Technologies: Hadoop, Hive

Education

Master, Naval Architecture and Ocean Engineering

Shanghai Ship and Shipping Research Institute

Bachelor, Hydraulic Engineering

Tianjin University

Technical Skills

Programming: Java, Python, C

Distributed Computing: Kafka, Flink, Spark, Hive, Hadoop

Storage: HDFS, Apache Doris, Hudi, Elasticsearch, S3

Infrastructure: Linux, Docker, Kubernetes

Platform Domains: Streaming, Analytical Computing, Privacy Computing, Metadata, Data Governance, Execution Planning

Engineering Philosophy

Infrastructure should simplify complexity. Reusable platform abstractions create more long-term value than one-off implementations. Architecture decisions should be documented with explicit trade-offs. Performance optimization requires measurable evidence, not assumptions.

Career Narrative

Scientific Computing → Analytical Computing → Streaming Infrastructure → Privacy Computing

Each transition deepened platform engineering scope: from batch analytical systems to real-time streaming at national scale, then to privacy-preserving distributed execution with query planning and storage abstraction.

Platform Engineering Patterns

Across three major platform builds, recurring architectural patterns emerged:

Layer Decoupling. Every platform separates ingestion, computation, storage, and serving into independent layers. Kafka decouples device producers from Flink at Jiayu. Hive decouples batch computation from Elasticsearch at Ping An. Data Service decouples execution engines from storage at Puxin.

Abstraction as Product. The most valuable platform artifact is never the pipeline — it is the abstraction layer. Metric Map, Kafka ingestion boundary, Data Service, and storage abstraction each compound in value as the platform grows.

Schema as Architecture. At 100K+ metric scale and 1M+ device scale, schema management requires dedicated architectural design. Dynamic schema expansion and platform-level metadata governance are not configuration tasks.

Measure Before Optimize. Benchmark platforms, consumer lag monitoring, and partition scan measurement precede optimization decisions. Assumption-based tuning at platform scale optimizes the wrong dimension confidently.

Selected Engineering Trade-offs

Platform	Decision	Benefit	Cost
MIRA	Dedicated query planner	Adaptive execution per query	Planner engineering complexity
MIRA	Storage abstraction	Backend portability	Interface indirection
Connected Vehicle	Kafka ingestion boundary	Independent scaling	Cluster operational overhead
Connected Vehicle	Doris serving layer	Analytical query performance	Flink-to-Doris data movement
Metric Platform	Hive + ES separation	Optimized per access pattern	Batch freshness latency
Metric Platform	Map-based dynamic schema	100K+ fields without migration	Translation layer complexity